SOFTWARE ENGINEERING · CHAPTER 07

Introduction to UML and Use Case Diagrams

Unified Modeling Language (UML) provides a standard notation for communicating software structure and behavior. Use case diagrams describe how external actors interact with a system.

UML 2.5.1 (OMG) **PlantUML · Mermaid · draw.io** **Running example: the online shop of Ch. 02-06**

AGENDA

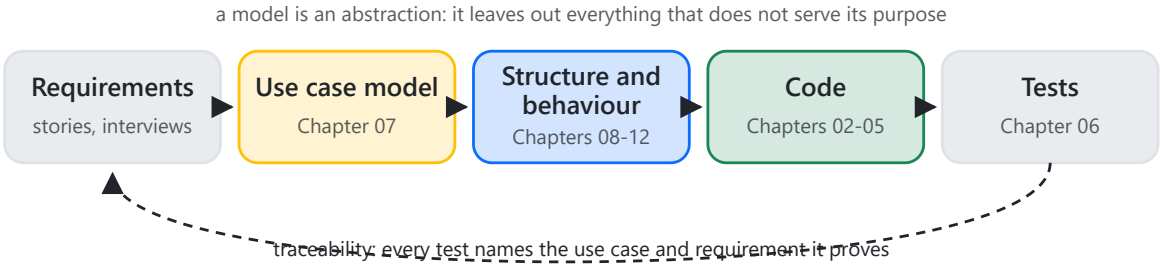
What we will cover

1. Purpose, notation, and limits of UML
2. Systems, actors, and use cases
3. Association, include, and extend relationships
4. Generalization between actors and use cases
5. System boundaries and scope
6. Primary and supporting actors
7. Use case names, goals, and scenarios
8. Preconditions, postconditions, and alternate flows
9. Business requirements and traceability
10. Common modeling mistakes and diagram clarity
11. Chapter Quiz
12. Lab-07

Click any item to jump directly to that topic.

From code to shared pictures: why model at all?

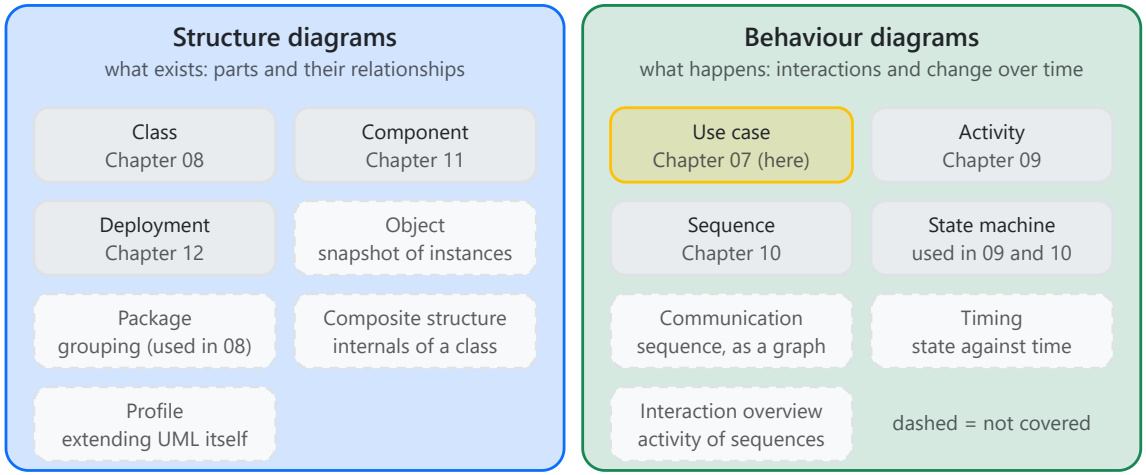
- Chapters 02-06 built the **online shop** bottom-up: Spring beans, JPA entities, REST endpoints, JWT security, JUnit and Mockito tests.
- Chapters 07-12 step back and ask the earlier question: **what** should the system do, **for whom**, and how do we agree on it before writing code?
- Prose is ambiguous and code is too detailed for that conversation. A **model** is a simplified, purposeful picture that a customer, a tester and a developer can all read.
- **UML** is the shared vocabulary for those pictures; the **use case diagram** is the first one you draw: who needs what from the system.
- Everything in this chapter is drawn as text (PlantUML) so diagrams live in git next to the code they describe.



What UML is: purpose and the two diagram families

- **UML 2.5.1** is an OMG standard (December 2017): a graphical *language* with defined symbols and meaning, not a method or a process.
- Three purposes: **communicate** (whiteboard sketch), **specify** (contract for a team), **document** (what was built).
- 14 diagram types in two families. **Structure** diagrams show what exists; **behaviour** diagrams show what happens over time.
- This course uses six: use case (07), class (08), activity (09), sequence (10), component (11), deployment (12).
- One rule for all of them: a diagram has **one audience and one question** it answers.

UML 2.5.1 · 14 diagram types in two families



TOPIC 1

Notation, tools, and the limits of UML

Mode (Fowler)	What the diagram is	In this course
Sketch	Selective, informal, thrown away after the talk	Whiteboard, first drafts
Blueprint	Complete enough for someone else to build from	Lab deliverables
Programming language	Executable model, code generated from it	Not used

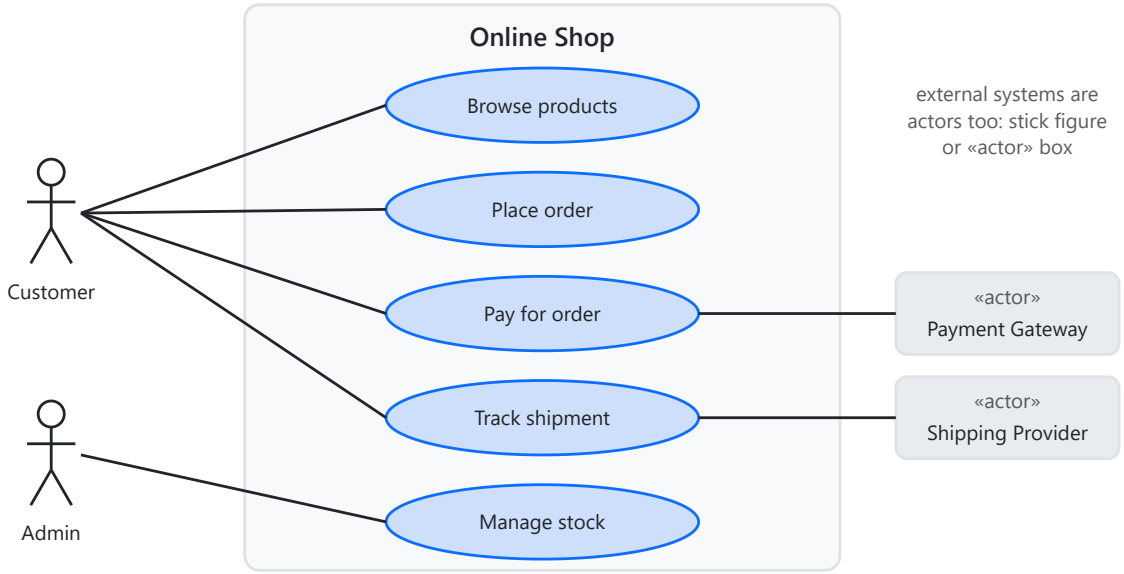
UML is	UML is not
A notation with defined semantics (metamodel)	A development process (that is RUP, Scrum, ...)
Tool-independent: text, whiteboard, draw.io	A guarantee of correctness or completeness
Good at structure and interaction	Good at user interface layout, data volume, performance
Understood across companies and languages	Meant to be drawn for every class you write

We write diagrams as **text**: PlantUML for use case, component and deployment diagrams; Mermaid where it has the notation (class, sequence, activity, state). Text diffs, reviews and versions like code.

```
@startuml shop-hello
' comments start with an apostrophe
' render: java -jar plantuml.jar -Playout=smetana shop-hello.puml
left to right direction
actor Customer
rectangle "Online Shop" {
  usecase "Place order" as UC2
}
Customer -- UC2
@enduml
```

⚠ **Limit to remember:** a use case diagram shows *who* and *what*, never *how* or *in which order*. Sequence and activity diagrams (Ch. 09-10) answer those questions. Do not force ordering into a use case diagram.

Systems, actors, and use cases: the first diagram



- **System**: the thing we build, drawn as a named rectangle (the subject).
- **Actor**: a *role* outside the system that interacts with it: a person, an organisation or another system. Never a job title, always a role ("Customer", not "Ann").
- **Use case**: a goal an actor achieves with the system, named as *verb + object*, delivering observable value.
- **Association**: the solid line saying "this actor takes part in this use case".

```
@startuml shop-v1
left to right direction
actor Customer
actor Admin
actor "Payment Gateway"   as PG <<system>>
actor "Shipping Provider" as SP <<system>>
rectangle "Online Shop" {
  usecase "Browse products" as UC1
  usecase "Place order"     as UC2
  usecase "Pay for order"   as UC3
  usecase "Track shipment"  as UC4
  usecase "Manage stock"    as UC5
}
Customer -- UC1
Customer -- UC2
Customer -- UC3
Customer -- UC4
Admin    -- UC5
UC3 -- PG
UC4 -- SP
@enduml
```

TOPIC 2

Notation elements and what they become in Java

Element	Symbol	Meaning	Java / Spring counterpart
Actor	Stick figure or box with «actor»	Role outside the system	Security role, API client, port interface
Use case	Ellipse, verb + object	Goal with observable value	Public service method or endpoint
Subject	Rectangle with the system name	What we build and test	The Spring Boot application
Association	Solid line	Actor participates	Controller mapping, `@PreAuthorize`
«include»	Dashed arrow base → included	Mandatory shared step	Method always called
«extend»	Dashed arrow extension → base	Optional, conditional step	`if` branch, optional collaborator
Generalization	Solid line, hollow triangle	Specialisation of actor or use case	Role hierarchy, sealed interface
Note	Folded corner, dashed anchor	Comment, extension point	Javadoc

A use case is a *promise*; the service method is where the promise is kept. One user-goal use case usually maps to one transactional service method.

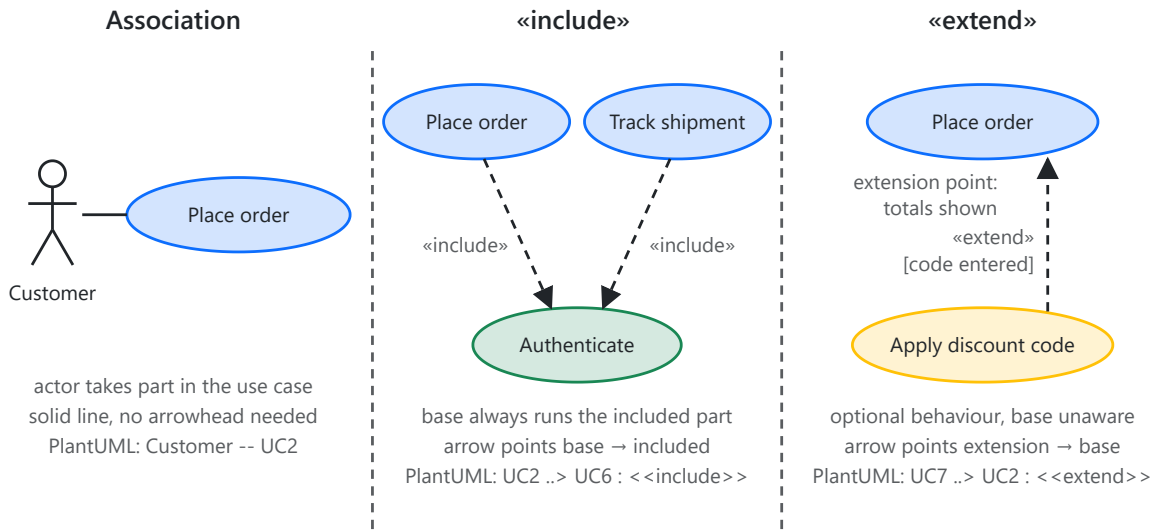
```java
/** UC-02 Place order · primary actor: Customer. */
@Service
public class OrderService {
    private final StockService stock;
    // supporting actor "Payment Gateway" seen through a port
    private final PaymentGateway gateway;
    private final OrderRepository orders;

    public OrderService(StockService stock, PaymentGateway gateway,
                        OrderRepository orders) {
        this.stock = stock;
        this.gateway = gateway;
        this.orders = orders;
    }

    /** One public method per user-goal use case. */
    @Transactional
    public Order placeOrder(CustomerId customer, Cart cart,
                            PaymentMethod method) {
        stock.reserve(cart);
        Receipt receipt = gateway.charge(cart.total(), method);
        return orders.save(Order.confirmed(customer, cart, receipt));
    }
}

/** UC-05 Manage stock · primary actor: Admin (a security role). */
@PreAuthorize("hasRole('ADMIN')")
public void adjustStock(ProductId id, int delta) {
    stock.adjust(id, delta);
}
```

Association, «include» and «extend» side by side




```
@startuml relationships
left to right direction
actor Customer
rectangle "Online Shop" {
  usecase "Place order"         as UC2
  usecase "Track shipment"      as UC4
  usecase "Authenticate"        as UC6
  usecase "Apply discount code" as UC7
}
' association: the actor takes part
Customer -- UC2
Customer -- UC4
' include: base ..> included (always)
UC2 ..> UC6 : <<include>>
UC4 ..> UC6 : <<include>>
' extend: extension ..> base (sometimes)
UC7 ..> UC2 : <<extend>>
note right of UC2
  extension points
  totals shown
end note
@enduml
```

Choosing between «include» and «extend», and what the code looks like

Question	«include»	«extend»
Does the base always need it?	Yes, every time	No, only under a condition
Who knows about whom?	Base names the included use case	Extension names the base; base only declares an extension point
Arrow direction	Base → included	Extension → base
Can the target stand alone?	Included: usually a subfunction, incomplete alone	Extension: a complete optional feature
Typical use	Authenticate, Validate address	Apply discount, Gift-wrap order

⚠ **Pitfall:** «include» and «extend» are not a way to decompose a use case into steps. If you have more includes than associations, you are drawing a flowchart, not a use case diagram.

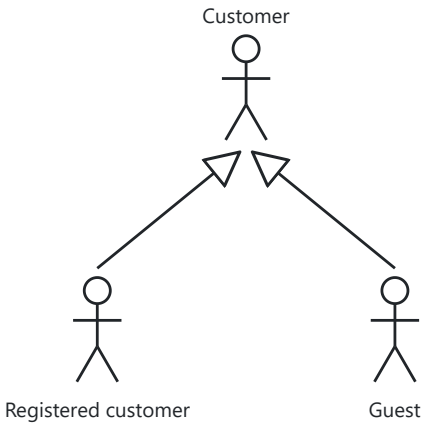
```
@Service
public class OrderService {
    private final AuthenticationService auth;   // «include» Authenticate
    private final DiscountPolicy discounts;     // «extend» Apply discount
    private final PricingService pricing;
    private final ShippingProvider shipping;
    private final OrderRepository orders;

    @Transactional
    public Order placeOrder(Cart cart, PaymentMethod method,
                            Optional<String> discountCode) {
        // included behaviour: runs on every execution of the base
        Customer customer = auth.requireCustomer();
        Totals totals = pricing.totalsFor(cart);
        // extension point "totals shown": only when the condition holds
        if (discountCode.isPresent()) {
            totals = discounts.apply(totals, discountCode.get());
        }
        return confirm(customer, cart, totals, method);
    }

    public Shipment trackShipment(OrderId id) {
        // the same included behaviour, reused by a second base
        Customer customer = auth.requireCustomer();
        return shipping.statusOf(orders.ownedBy(customer, id));
    }
}
```

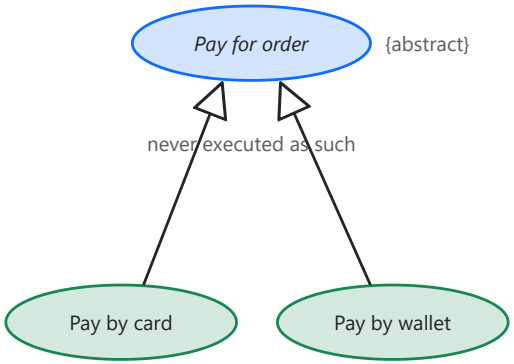
Generalization between actors and between use cases

Actor generalization



children inherit every association of Customer

Use case generalization



children specialise the parent's scenario and goal

- **Actor generalization**: the child plays every role of the parent plus its own. *Registered customer* can do everything a *Customer* can, and also *Place order*.
- **Use case generalization**: the child follows the parent's goal and main flow but replaces or refines steps. Mark the parent *{abstract}* if it is never run as such.
- Symbol: solid line with a **hollow triangle** at the general element (same as class inheritance in Ch. 08).

```
@startuml generalization
left to right direction
actor Customer
actor "Registered customer" as Reg
actor Guest
Reg   --|> Customer
Guest --|> Customer
rectangle "Online Shop" {
  usecase "Browse products" as UC1
  usecase "Place order"     as UC2
  usecase "Pay for order"   as Pay
  usecase "Pay by card"     as Card
  usecase "Pay by wallet"   as Wallet
}
Customer -- UC1
Reg      -- UC2
Reg      -- Pay
Card   --|> Pay
Wallet --|> Pay
@enduml
```

Generalization in code: sealed types and role hierarchies

```java
// Use case generalization: one abstract goal, several concrete ways
public sealed interface PaymentMethod permits CardPayment, WalletPayment {}
public record CardPayment(String token) implements PaymentMethod {}
public record WalletPayment(String walletId) implements PaymentMethod {}

@Service
public class PaymentService {
    private final PaymentGateway gateway;
    private final WalletClient wallet;

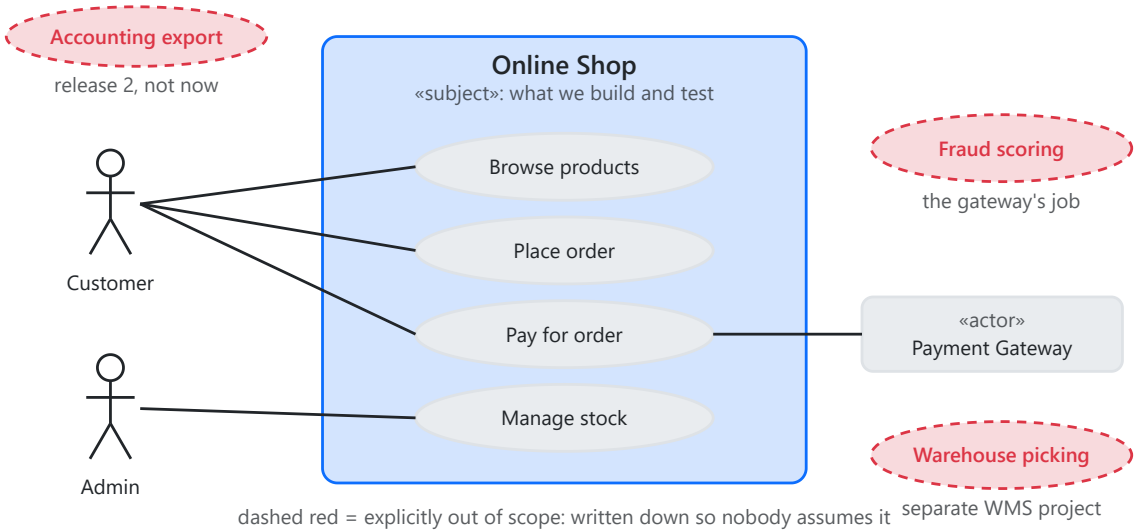
    /** UC-03 Pay for order: the parent scenario; a child is chosen at run time. */
    public Receipt pay(Order order, PaymentMethod method) {
        return switch (method) {
            case CardPayment c   -> gateway.charge(order.total(), c.token());
            case WalletPayment w -> wallet.debit(w.walletId(), order.total());
        };
    }
}

// Actor generalization: roles inherit permissions (Spring Security 7, Ch. 05)
@Bean
RoleHierarchy roleHierarchy() {
    return RoleHierarchyImpl.fromHierarchy("ROLE_REGISTERED > ROLE_CUSTOMER");
}
```

Use generalization when	Avoid it when
Several actors share most associations and differ in a few	You only want to say "both can do X" (just draw two lines)
Two use cases share goal, trigger and most steps, but a decisive step differs	The difference is one `if` inside a step (use an extension or a plain alternate flow)
The parent is a real concept stakeholders use ("Customer", "Pay")	The parent exists only to save arrows

Java's `sealed` types mirror use case generalization exactly: the set of children is closed and the `switch` must handle each of them, just as each child use case needs its own scenario.

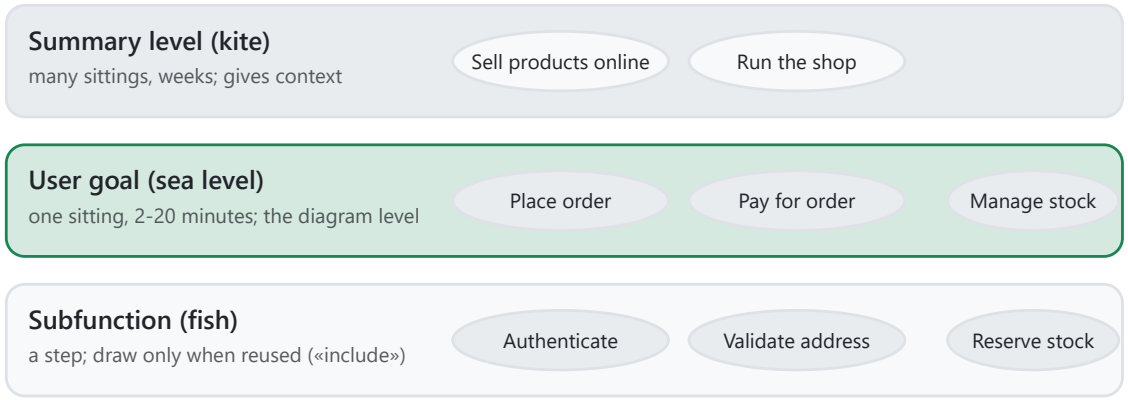
System boundaries: what is inside, what is outside, what is out of scope



- The **boundary rectangle** answers "what are we responsible for?" Everything inside gets designed, coded and tested by us; everything outside is somebody else's system or a person.
- Actors are **always outside**. A database, a scheduler or a framework we control is *inside* and is never an actor.
- Scope is a **decision**, not a discovery: write the out-of-scope list next to the diagram, otherwise everyone assumes their favourite feature is in.
- Move the boundary and the actors change: if the warehouse were inside, "Warehouse worker" would become an actor and "Pick order" a use case.

```
rectangle "Online Shop" as sys {
  usecase "Pay for order" as UC3
}
actor "Payment Gateway" as PG <<system>>
UC3 -- PG
note bottom of PG
  fraud scoring, 3-D Secure:
  the gateway's problem, out of scope
end note
```

Scope in two dimensions: system boundary and goal level

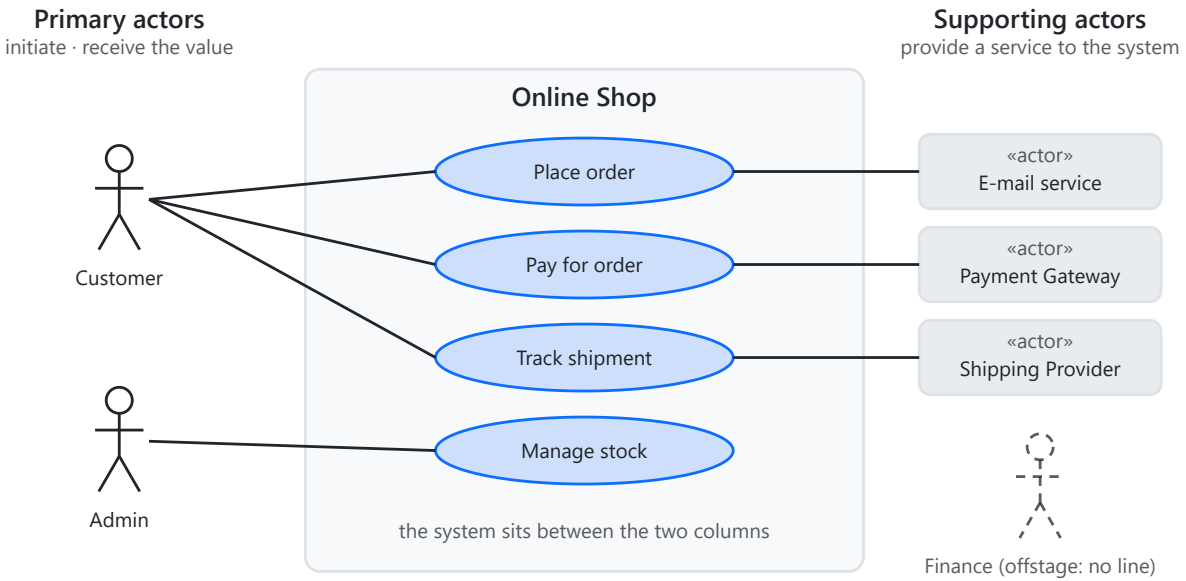


Cockburn's levels: keep one diagram at one level; kites for scope, fish only when shared

Level	Test	Example
Summary	Why does the actor do the user goals?	Sell products online
User goal	Can the actor go for lunch happy afterwards?	Place order
Subfunction	Is it a step of a user goal?	Validate address

```
Scope statement · Online Shop v1
IN  : browse, cart, order, pay (card, wallet),
      track shipment, admin stock management
OUT : warehouse picking (WMS), accounting export,
      fraud scoring (gateway), marketing e-mails
Actors: Customer (guest, registered), Admin,
        Payment Gateway, Shipping Provider
Level : user goals in the main diagram;
        subfunctions only when included twice
```

Primary actors on the left, supporting actors on the right



- **Primary actor**: has the goal, triggers the use case, receives the value. Drawn on the **left**.
- **Supporting (secondary) actor**: is asked by the system for a service during the use case. Drawn on the **right**. Almost always another system.
- **Offstage stakeholder**: has an interest (Finance wants every payment recorded) but never interacts. Listed in the specification, not connected in the diagram.

```
@startuml actors
left to right direction
' primary actors: declared first, drawn left
actor Customer
actor Admin
rectangle "Online Shop" {
  usecase "Place order"    as UC2
  usecase "Pay for order"  as UC3
  usecase "Track shipment" as UC4
  usecase "Manage stock"   as UC5
}
' supporting actors: declared last, drawn right
actor "E-mail service"    as Mail <<system>>
actor "Payment Gateway"   as PG   <<system>>
actor "Shipping Provider" as SP   <<system>>
Customer -- UC2
Customer -- UC3
Customer -- UC4
Admin    -- UC5
UC2 -- Mail
UC3 -- PG
UC4 -- SP
@enduml
```

A supporting actor becomes a port: interface in production, mock in tests

	Primary	Supporting	Offstage
Starts the use case	Yes	No	No
Interacts during it	Yes	Yes, when asked	No
Gets the value	Yes	No (paid to help)	Indirectly
Position	Left	Right	Spec only
Shop example	Customer, Admin	Payment Gateway	Finance, Legal
In Java	Controller caller, role	Port interface + adapter	Audit log, report

💡 The same actor can be primary in one use case and supporting in another: the Shipping Provider is supporting in *Track shipment* but primary in *Report delivery* when it calls our webhook.

```java
// The supporting actor is seen through a port: an interface the system calls
public interface PaymentGateway {
    Receipt charge(Money amount, PaymentMethod method);
}

// Production: a REST adapter (Ch. 04). Tests: a Mockito double (Ch. 06).
@ExtendWith(MockitoExtension.class)
class PlaceOrderTest {
    @Mock PaymentGateway gateway;             // the supporting actor
    @Mock StockService stock;
    @Mock OrderRepository orders;
    @InjectMocks OrderService service;

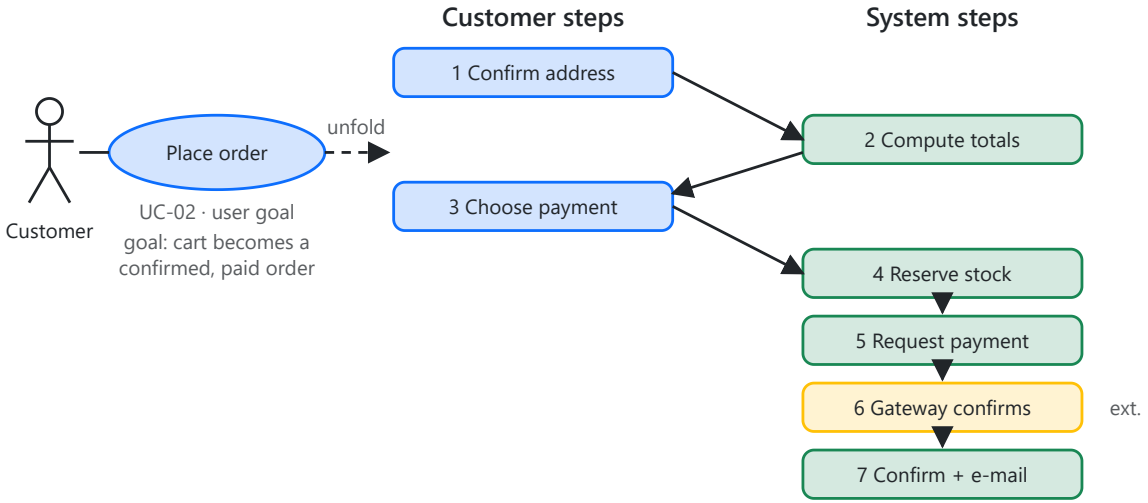
    @Test @DisplayName("UC-02 Place order: main success scenario")
    void confirmsOrderWhenGatewayAccepts() {
        when(gateway.charge(any(), any())).thenReturn(new Receipt("ok-1"));
        when(orders.save(any())).thenAnswer(a -> a.getArgument(0));

        Order order = service.placeOrder(new CustomerId(7), cart(),
                                         new CardPayment("tok"));

        assertThat(order.status()).isEqualTo(Status.CONFIRMED);
        verify(gateway).charge(eq(Money.of("39.90")), any());
    }
}
```

Use case names, goals, and scenarios: the ellipse is only the title

- **Name** = active *verb* + *object*, from the actor's point of view: *Place order*, not *Order processing* or *OrderService*.
- **Goal**: the one sentence of value the primary actor wants. If you cannot state it, it is not a use case.
- **Scenario**: one path through the use case, a numbered sequence of actor and system steps. The **main success scenario** is the path where nothing goes wrong.
- A use case is the *set* of all its scenarios (success and failure). The diagram shows the set; the specification writes the scenarios.
- Each step is a sentence: *who* does *what*, in the present tense, without UI details ("Customer chooses a payment method", not "Customer clicks the blue button").



main success scenario: the actor reaches the goal; step 6 is done by the supporting actor

Good and bad names, and the header of a use case specification

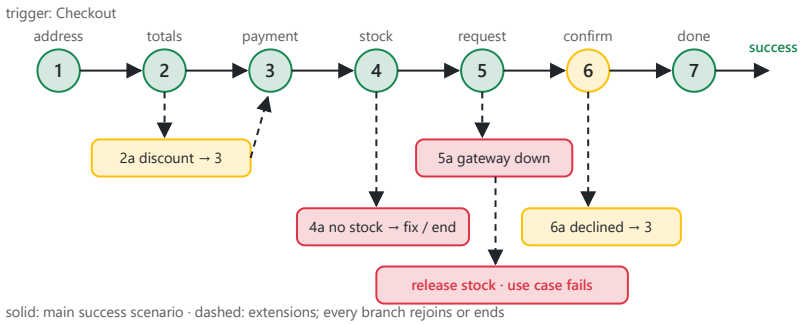
Bad name	Why	Better
Order	Noun only, no goal	Place order
Click "Pay" button	UI step, not a goal	Pay for order
Order processing	System-internal view	Fulfil order (if in scope)
Login	Nobody's goal; a subfunction	Authenticate («include»)
Manage everything	Too big, several goals	Manage stock · Manage catalogue
OrderService.placeOrder	Implementation name	Place order

💡 Cockburn's fully dressed template (right) is the industry default. Fill the header first: if *Goal* and *Primary actor* are hard to write, the ellipse is wrong.

```
USE CASE UC-02  Place order
Scope         : Online Shop (system use case)
Level         : User goal (sea level)
Primary actor: Registered customer
Goal          : Turn the shopping cart into a confirmed, paid order
Stakeholders : Customer - correct order, fast checkout
               Shop     - payment secured before fulfilment
               Warehouse- a pickable order with reserved stock
Precondition : Customer is authenticated; cart holds at least one item
Minimal guar.: No order exists without payment; cart is unchanged
Success guar.: Order CONFIRMED, stock reserved, payment captured,
               confirmation e-mail queued
Trigger       : Customer selects "Checkout"
Frequency     : about 2 000 per day, peaks before holidays
Open issues   : partial shipment when one line is out of stock?
```

Main success scenario, extensions, and the guarantees around them

```
Main success scenario
  1. Customer confirms the delivery address
  2. System computes totals (items, shipping, tax) and shows them
  3. Customer chooses a payment method (card or wallet)
  4. System reserves stock for every order line
  5. System requests payment from the Payment Gateway
  6. Payment Gateway confirms the payment
  7. System marks the order CONFIRMED and queues the e-mail
Extensions (alternate flows)
  2a. Customer enters a discount code:
      2a1. System validates the code and recalculates totals; -> 3
  4a. A line is out of stock:
      4a1. System shows the line; Customer removes it (-> 4) or cancels
  5a. Payment Gateway unreachable:
      5a1. System retries twice; 5a2. releases stock; use case fails
  6a. Payment declined:
      6a1. System releases stock and offers another method; -> 3
Postconditions: success guarantee, or minimal guarantee on failure
```



solid: main success scenario · dashed: extensions; every branch rejoins or ends

- **Preconditions** are guaranteed before the use case starts and are *not* re-checked in the steps.
- **Postconditions**: the *success guarantee* after the main scenario, the *minimal guarantee* after any failure.
- **Extensions** are numbered after the step they branch from (2a, 4a) and say where they rejoin or that the use case fails.
- Every extension is a **test case** waiting to be written (next slide).

From scenario to Gherkin and to code: conditions become guards and asserts

```
Feature: UC-02 Place order

  Background:
    Given registered customer "ann" is signed in

  Scenario: Main success scenario
    Given her cart holds 2 x "USB-C cable" in stock
    When she checks out paying by card
    Then the order is CONFIRMED
    And stock of "USB-C cable" is reduced by 2
    And a confirmation e-mail is queued for "ann"

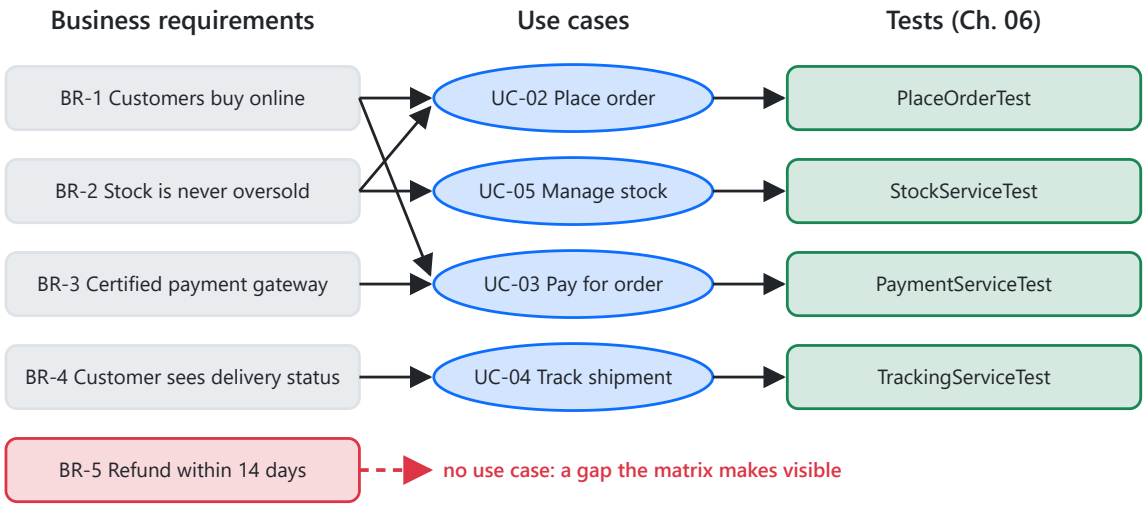
  Scenario: 6a Payment declined
    Given her cart holds 1 x "Keyboard" in stock
    When she checks out with a card the gateway declines
    Then no order exists for "ann"
    And the reserved stock is released
```

Given = preconditions, *When* = trigger and actor steps, *Then* = postconditions. One Gherkin scenario per use case scenario; the extension id stays in the title.

```java
@Transactional
public Order placeOrder(Cart cart, PaymentMethod method) {
    // preconditions: guaranteed by the caller, asserted once,
    // never "handled" as an alternate flow
    Customer customer = auth.requireCustomer();
    if (cart.isEmpty()) {
        throw new IllegalStateException("precondition: cart not empty");
    }
    stock.reserve(cart);                                    // step 4
    try {
        Receipt receipt = gateway.charge(cart.total(), method);  // 5-6
        Order order = Order.confirmed(customer, cart, receipt);  // 7
        mail.queueConfirmation(order);
        return orders.save(order);
    } catch (PaymentDeclinedException | GatewayUnavailableException e) {
        stock.release(cart);       // 5a / 6a: minimal guarantee holds
        throw e;
    }
}
```

Specification	Code	Test
Precondition	Guard clause, `@PreAuthorize`	`Given`, fixture
Main scenario step	Statement in the service method	`When`
Success guarantee	Return value, saved state	`Then`, `assertThat`
Extension	`catch`, `if`, strategy	Second scenario, `verify`

Business requirements and traceability: requirement → use case → test



every requirement has at least one use case; every use case has at least one test; anything else is a question

- **Business requirement** (BR): why the sponsor pays; stated as an outcome, not a feature. "Stock is never oversold", not "add a reserve button".
- **Use case**: how an actor gets that outcome with the system. One BR maps to one or more use cases; one use case may serve several BRs.
- **Test**: proof that the use case works, one test class per use case, one method per scenario.
- **Traceability** is the ability to follow these links in both directions: forward to check coverage, backward to explain why a test or class exists.
- Two questions the matrix answers instantly: *which requirement has no use case?* and *which use case has no test?*

💡 Non-functional requirements (response time, GDPR) are not use cases. Attach them to the use case they constrain in the specification's "Special requirements" field.

Traceability without a tool: ids in names, tags and a matrix in the repository

```java
// One test class per use case; one method per scenario; ids in the names
@Tag("UC-02") @Tag("BR-1") @Tag("BR-2")
@DisplayName("UC-02 Place order")
@ExtendWith(MockitoExtension.class)
class PlaceOrderTest {

    @Test @DisplayName("main success scenario: order CONFIRMED, stock reserved")
    void mainSuccessScenario() {
        // Given ... When ... Then, as in the Gherkin scenario
    }

    @Test @DisplayName("2a discount code recalculates totals before payment")
    void extension2aDiscountCode() { }

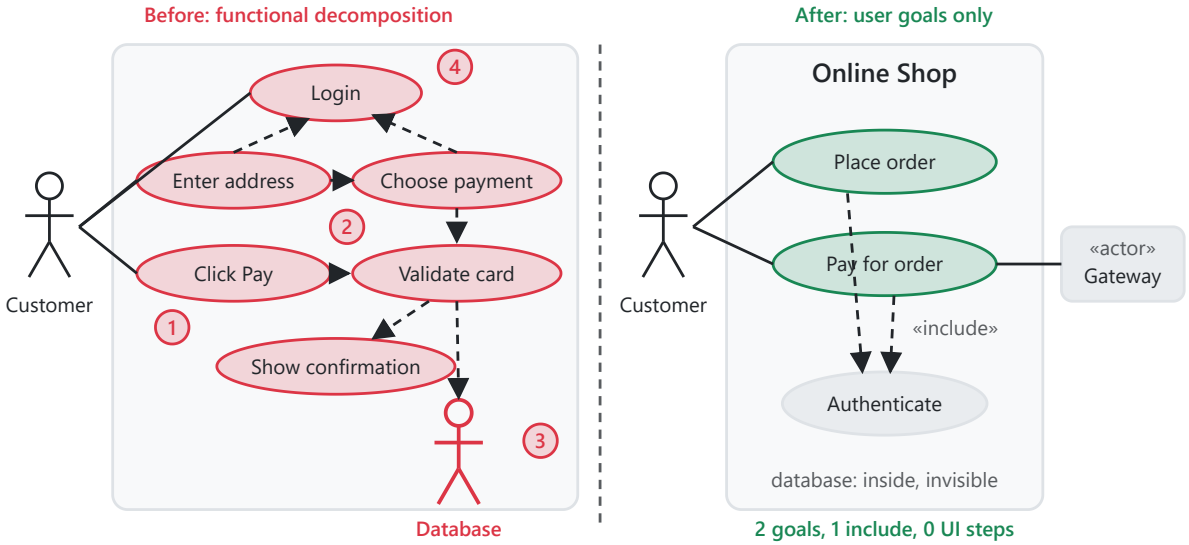
    @Test @DisplayName("6a payment declined releases stock, no order saved")
    void extension6aPaymentDeclined() {
        when(gateway.charge(any(), any()))
            .thenThrow(new PaymentDeclinedException("insufficient funds"));
        assertThatThrownBy(() -> service.placeOrder(cart(), card()))
            .isInstanceOf(PaymentDeclinedException.class);
        verify(stock).release(any());
        verify(orders, never()).save(any());
    }
}
```

```bash
# run only the tests that prove one use case or one requirement
./mvnw test -Dgroups=UC-02
./mvnw test -Dgroups=BR-2
```

Requirement	Use case	Test (scenario)
BR-1 buy online	UC-02 Place order	PlaceOrderTest.mainSuccessScenario
BR-1 buy online	UC-03 Pay for order	PaymentServiceTest.paysByCard, paysByWallet
BR-2 never oversold	UC-02 (4a), UC-05	PlaceOrderTest.extension4aOutOfStock, StockServiceTest
BR-3 certified gateway	UC-03	PaymentGatewayContractTest
BR-4 delivery status	UC-04 Track shipment	TrackingServiceTest
BR-5 refunds	none	none: raise with the product owner

- Keep the matrix as `docs/traceability.md` next to `docs/use-cases/*.puml`; review it in the same merge request as the code.
- Ids never change once published (UC-02 stays UC-02 even if renamed).
- JUnit 6 `@Tag` plus Maven Surefire `-Dgroups` gives a free "run the tests of this requirement" button.

Common modeling mistakes: the same checkout, before and after



1. **UI steps as use cases** ("Click Pay", "Show confirmation"): steps belong in the scenario, not in ellipses. Ask: is this someone's goal?
2. **Functional decomposition**: chaining ellipses with «include» to show order. Order is what activity and sequence diagrams are for.
3. **Internal parts as actors**: the database, a scheduler or a service class are inside the boundary. Actors are external roles only.
4. **Login as a goal**: nobody logs in for its own sake. It is a subfunction, included by the real goals.
5. **Missing boundary or scope list**: without the rectangle nobody can tell what is being built.
6. **Wrong arrow direction** on «extend» (base → extension) and generalization triangles at the child.

⚠ Rule of thumb: **5 to 12 use cases per diagram**. More means either wrong level (fish) or a system that needs packages (one diagram per subsystem).

Checklist for a good use case diagram

1 Boundary named and drawn

The rectangle carries the system name. Every ellipse is inside; every actor is outside. An out-of-scope list sits next to the diagram.

2 Every use case is a goal

Verb + object, sea level, stated from the actor's point of view. You can write its Goal line and name its primary actor in one breath.

3 Actors are external roles

Primary on the left, supporting on the right, external systems marked «system». No database, no internal component, no person's name.

4 Relationships are rare and right

Associations outnumber includes and extends. «include» points base → included, «extend» points extension → base with a condition, triangles point at the parent.

5 Readable in ten seconds

5 to 12 ellipses, no crossing lines where avoidable, one goal level, consistent naming, a legend only if you used stereotypes.

6 Backed by text and ids

Every ellipse has an id (UC-nn) and a specification; the source (.puml) and the export (.png/.svg) are both in git; the traceability matrix references the ids.

```
' Layout hints that make PlantUML diagrams readable
left to right direction          ' actors left/right, not top/bottom
skinparam packageStyle rectangle ' plain boundary, no folder tab
skinparam linetype ortho         ' straight connectors, fewer crossings
actor Customer                   ' declaration order = drawing order
actor Admin
rectangle "Online Shop" { ... }
actor "Payment Gateway" as PG <<system>>
Customer -[hidden]down- Admin    ' pins Admin below Customer
```

☑ **Review ritual:** read the diagram aloud as sentences ("The Customer places an order, which includes authenticating"). Any sentence that sounds wrong marks a modelling error.

BEST PRACTICES

A use-case-driven workflow that fits the rest of this course

Start with actors, not features

List who and what touches the system. Each actor gets goals; goals become use cases. Features without an actor are questions for the sponsor.

Diagram first, then specify the risky ones

The diagram is cheap. Fully dressed specifications cost hours: write them for the use cases with money, security or many extensions (Place order, Pay), keep the rest casual.

Text in git, pictures generated

Commit `.puml` sources under `docs/`; render in CI. A diagram that cannot be diffed will not be maintained.

Let the use case shape the code

One user-goal use case → one transactional service method; supporting actors → ports; extensions → exceptions or strategies. Ch. 08 and 10 refine this.

One scenario, one test

Main scenario and each extension become JUnit methods or Gherkin scenarios tagged with the use case id. Coverage then means "scenarios proven", not "lines executed".

Revisit on every change request

A new request is either a new use case, a new extension or a changed step. Deciding which, before coding, keeps the matrix and the tests honest.

Chapter quiz 10 questions

1. What is UML 2.5.1 best described as?

- A software development process
- A graphical modelling language standardised by the OMG
- A Java code generator
- A testing framework

2. Which of these is a UML structure diagram?

- Sequence diagram
- Activity diagram
- Component diagram
- Use case diagram

3. In the online shop, how should the external Payment Gateway appear on the use case diagram?

- As a use case inside the boundary
- As a supporting actor outside the boundary, on the right
- As a note attached to Pay for order
- It should not appear at all

4. What does UC2 ..> UC6 : <<include>> mean?

- UC6 optionally adds behaviour to UC2
- UC2 always runs the behaviour of UC6 as part of itself
- UC6 is a specialisation of UC2
- UC2 and UC6 share an actor

5. In which direction does the «extend» dashed arrow point?

- From the base use case to the extension
- From the extension to the base use case
- From the actor to the extension
- Either direction is acceptable

6. Which is the best use case name?

- Order
- Click the Pay button
- Place order
- OrderService.placeOrder

7. Which statement about preconditions is correct?

- They are checked again in step 1 of the main scenario
- They describe the state after the use case succeeds
- They must be true before the use case starts and are not re-checked in the steps
- They list the supporting actors

8. What does a solid line with a hollow triangle between Registered customer and Customer express?

- Registered customer includes Customer
- Registered customer is a specialisation of Customer and inherits its associations
- Customer extends Registered customer
- They are the same actor drawn twice

9. What does a traceability matrix in this chapter link together?

- Classes, packages and modules
- Business requirements, use cases and tests
- Actors, screens and database tables
- Sprints, stories and developers

10. A diagram shows Login, Enter address, Click Pay and Show confirmation as use cases chained with «include». What is the main mistake?

- Functional decomposition: UI steps drawn as use cases instead of one user goal
- Too few actors
- Missing generalization
- The system boundary is too small

WRAP-UP

Summary

UML 2.5.1 is a notation with 14 diagram types in two families (structure, behaviour); it is not a method. We use six of them in Ch. 07-12.

A use case diagram answers "who needs what from the system": actors (external roles) outside a named boundary, goals (verb + object) inside.

Association = participates. «include» = always, arrow base → included. «extend» = optional under a condition, arrow extension → base.

Generalization (hollow triangle at the parent) lets child actors inherit associations and child use cases specialise a goal; in Java: role hierarchies and sealed types.

The boundary is a decision: write the out-of-scope list. Keep one diagram at one goal level (sea level), 5 to 12 use cases.

Primary actors (left) start the use case and get the value; supporting actors (right) provide a service and become ports, mocked in tests.

The ellipse is only the title: a Cockburn specification gives goal, actor, preconditions, main success scenario, extensions and guarantees.

Preconditions become guards, steps become service code, guarantees become asserts; extensions become catch blocks and extra test scenarios (Gherkin).

Traceability BR → UC → test with stable ids, JUnit tags and a matrix in git exposes gaps in both directions.

Open Lab-07: 5 tasks + 1 challenge

Next chapter: 08 · Class Diagrams